

Day 117 Part-3 — MLflow Practical Notes for Beginners

These notes extend the basic Day 117 Part-3 lesson. The goal is to learn **how MLflow becomes useful when you actually start experimenting with models**.

The central idea is:

```
Train one model
  ↓
Record what happened
  ↓
Change something
  ↓
Train again
  ↓
Compare the runs
  ↓
Understand which changes helped
```

MLflow Tracking is designed around exactly this idea: runs can store parameters, metrics, models, artifacts, and other metadata, and related runs can be grouped into experiments.

1. Compare Different Models in One Experiment

Suppose your problem is:

Predict whether a customer will purchase a product.

You could create one MLflow experiment:

```
Experiment:
Customer Purchase Prediction
```

Then create several runs:

```
Run 1
→ Logistic Regression

Run 2
→ Decision Tree

Run 3
→ Random Forest
```

Each run uses the same basic dataset and prediction problem.

You could record:

Run	Model	Accuracy	F1
1	Logistic Regression	0.81	0.79
2	Decision Tree	0.83	0.81
3	Random Forest	0.86	0.84

This is much more useful than simply seeing:

```
accuracy = 0.86
```

because MLflow lets you retain the context of **which model produced that result**.

Beginner mental model

```
Same problem
  ↓
Different models
  ↓
Different MLflow runs
  ↓
Compare metrics
```

What should you log?

For each run:

```
model_name
parameters
accuracy
precision
recall
F1
trained model
```

2. Track Hyperparameters

A **hyperparameter** is a setting you choose before or during model training.

For example, a Decision Tree has settings such as:

```
max_depth
min_samples_split
```

A Random Forest may have:

```
n_estimators
max_depth
```

Logistic Regression can have settings such as:

```
C
solver
max_iter
```

Suppose you keep the model the same but change:

```
max_depth
```

Your MLflow runs might look like:

Run	max_depth	Accuracy
1	2	0.78
2	4	0.84
3	6	0.82

Now MLflow helps answer:

Which hyperparameter setting produced which result?

Conceptually:

```
Parameter:
max_depth = 4

Metric:
accuracy = 0.84
```

MLflow distinguishes parameters from metrics, and its run-search functionality can filter runs using either. :chatgpt-content-reference{index="1"}

Remember

```
Hyperparameter
= something I choose

Metric
= something I observe
```

3. Track Multiple Evaluation Metrics

Do not always record only:

```
accuracy
```

For classification, depending on the problem, you might track:

```
accuracy
precision
recall
F1
```

For regression:

```
MAE
MSE
RMSE
R2
```

Imagine:

Model	Accuracy	Precision	Recall	F1
Logistic Regression	0.88	0.84	0.76	0.80
Decision Tree	0.86	0.79	0.85	0.82

Looking only at accuracy would hide some of the differences.

MLflow allows multiple metrics to be logged for the same run. :chatgpt-content-reference{index="2"}

Conceptually:

```
Run
├─ accuracy = 0.88
├─ precision = 0.84
├─ recall = 0.76
└─ F1 = 0.80
```

4. Experiment with Class Weights

This is especially useful when your target classes are **imbalanced**.

Suppose:

```
900 customers → No Fraud
100 customers → Fraud
```

The classes are not balanced.

Some classifiers allow a setting such as:

```
class_weight="balanced"
```

You could compare:

```
Run 1
class_weight = None

Run 2
class_weight = balanced
```

Then log metrics such as:

```
precision
recall
F1
```

The MLflow comparison might conceptually become:

Run	class_weight	Accuracy	Recall	F1
1	None	0.92	0.51	0.61
2	balanced	0.88	0.79	0.70

Notice something important:

```
Higher accuracy
```

doesn't automatically mean:

```
better for every objective
```

The weighted version could reduce accuracy while improving recall on the minority class.

Important distinction

Don't confuse **class weights** with the internal learned weights of a model.

For this lesson:

```
class_weight
→ setting you choose
→ parameter/hyperparameter

learned coefficients or model weights
→ learned during training
```

For a beginner MLflow exercise, track `class_weight` as a **parameter**.

5. Log the Trained Model

Suppose you train:

```
Random Forest
accuracy = 0.87
```

You may later want the actual trained model—not just its accuracy.

MLflow can record trained models associated with runs. Its scikit-learn integration can also log models automatically. :chatgpt-content-reference{index="3"}

Conceptually:

```
Run 7
├── Parameters
│   ├── n_estimators = 100
│   └── max_depth = 5
├── Metrics
│   └── F1 = 0.86
└── Model
    └── trained Random Forest
```

This matters because:

```
metric
→ tells you how it performed

trained model
→ lets you actually use that fitted model
```

6. Log Artifacts

An **artifact** is an output file associated with a run.

Useful beginner artifacts include:

```
confusion_matrix.png
feature_importance.png
predictions.csv
evaluation_report.txt
```

Suppose your run produces:

```
accuracy = 0.87
```

You could also save:

```
confusion_matrix.png
```

Then the run conceptually contains:

```
Run 4
├── Parameters
├── Metrics
├── Model
└── Artifacts
    ├── confusion_matrix.png
    └── predictions.csv
```

MLflow's artifact store is intended for outputs such as model files, images, and other data files. :chatgpt-content-reference{index="4"}

Why is this useful?

A number tells you:

```
F1 = 0.82
```

but a confusion matrix can help you understand:

```
How many positive examples were missed?
How many false alarms occurred?
```

So metrics and artifacts complement each other.

7. MLflow Autologging

Initially, learning manual logging is valuable:

```
start run
↓
log parameter
↓
train
↓
calculate metric
↓
log metric
↓
log model
```

But MLflow can automate much of this.

For scikit-learn, you can enable:

```
mlflow.sklearn.autolog()
```

Current MLflow documentation says scikit-learn autologging can capture model parameters, metrics, the fitted estimator/model, and related metadata automatically; it also works with scikit-learn Pipeline objects. :chatgpt-content-reference{index="5"}

Conceptually:

```
Enable MLflow autologging
    ↓
Train normal sklearn model
    ↓
MLflow automatically records
many useful details
```

Instead of manually writing:

```
log C
log max_iter
log accuracy
log model
...
```

MLflow can capture much of that information for you.

Beginner recommendation

Learn in this order:

```
1. Manual logging
    ↓
Understand parameter/metric/run concepts

2. Autologging
    ↓
Reduce repetitive logging code
```

8. Compare Runs in the MLflow UI

This is one of the most useful practical parts of MLflow.

Imagine you have:

```
20 runs
```

You don't want to manually open Python files and compare them.

The MLflow Tracking UI can list and compare runs, search based on parameter or metric values, visualize metrics, and inspect run artifacts. :chatgpt-content-reference{index="6"}

You might see something conceptually like:

Run	Model	max_depth	class_weight	F1
run_A	Logistic	—	None	0.78
run_B	Tree	3	None	0.80
run_C	Tree	5	None	0.82
run_D	Tree	5	balanced	0.86

Now you can investigate:

What changed between run_C and run_D?

Answer:

class_weight

and then inspect how the metrics changed.

This is the real power of experiment tracking

Not:

"What was my highest number?"

but:

"What did I change,
and how did the result change?"

9. Use Run Names and Tags

After many runs, names such as:

Run 1
Run 2
Run 3
Run 4

aren't very descriptive.

Instead, names could be:

logistic_baseline

tree_depth_5

rf_100_trees

```
rf_balanced
```

You can also add **tags**.

Tags provide extra context about a run and can be searched in MLflow. :chatgpt-content-reference{index="7"}

For example:

```
stage = experiment
task = classification
purpose = baseline
dataset_version = v1
```

Conceptually:

```
Run:
rf_balanced

Parameters:
n_estimators = 100
class_weight = balanced

Tags:
purpose = imbalance_test
dataset = customer_v1
```

Parameter vs tag

A simple beginner distinction:

```
Parameter
→ affects or describes model configuration

Tag
→ describes/contextualizes the run
```

For example:

```
max_depth = 5
→ parameter

purpose = "baseline"
→ tag
```

10. Reproduce a Previous Run

Suppose two weeks later you find:

Run 37

```
model = Random Forest
n_estimators = 200
max_depth = 6
class_weight = balanced
```

F1 = 0.88

Without tracking, you might think:

What settings did I use again?

MLflow keeps run metadata, parameters, metrics, timestamps, artifacts, and potentially model information together, making it much easier to reconstruct what happened. :chatgpt-content-reference{index="8"}

Conceptually:

```
Old run
  ↓
Inspect recorded parameters
  ↓
Use same data/preprocessing/model settings
  ↓
Run experiment again
```

This leads to an important concept:

Reproducibility

You should ideally be able to answer:

```
Which dataset?

Which preprocessing?

Which model?

Which hyperparameters?

Which code/version?

Which metric?

Which model artifact?
```

MLflow helps record important parts of that history.

It doesn't automatically guarantee perfect reproducibility—you still need disciplined data and code management—but it makes the process much easier.

11. Track Preprocessing + Pipeline Experiments

This connects directly to **Day 117**.

Suppose your pipeline is:

```
Missing-value handling
      ↓
Encoding
      ↓
Scaling
      ↓
Logistic Regression
```

You could test preprocessing choices.

Run 1

```
Missing values:
mean

Scaling:
StandardScaler

Model:
Logistic Regression
```

Run 2

```
Missing values:
median

Scaling:
StandardScaler

Model:
Logistic Regression
```

Run 3

```
Missing values:
median

Scaling:
none
```

Model:
Logistic Regression

Then compare:

Run	Imputer	Scaling	Model	F1
1	Mean	Yes	Logistic	0.80
2	Median	Yes	Logistic	0.83
3	Median	No	Logistic	0.79

Now you're tracking more than the model.

You're tracking the **entire ML workflow**.

This is important because:

Model performance

can depend on:

```
data
+
preprocessing
+
model
+
hyperparameters
```

not only on the model algorithm itself.

MLflow's scikit-learn autologging supports estimators including scikit-learn Pipelines, which makes this a natural workflow to practice. :chatgpt-content-reference{index="9"}

12. Beginner MLflow Practical Project

This is the best exercise to combine everything.

Problem

Create a small binary classification project:

Customer Purchase Prediction

Suppose the dataset contains:

```
Age
Income
City
```

```
Previous_Purchases
Purchased
```

Target:

```
Purchased
```

Step 1 — Build a baseline

Start with:

```
Preprocessing
  ↓
Logistic Regression
```

Create:

```
Run 1:
logistic_baseline
```

Track:

```
model_name
accuracy
precision
recall
F1
```

Step 2 — Try another model

Create:

```
Run 2:
decision_tree_baseline
```

Track:

```
model_name = DecisionTree

max_depth

accuracy
precision
recall
F1
```

Now you have:

Logistic Regression
vs
Decision Tree

Step 3 — Change a hyperparameter

Keep Decision Tree but change:

`max_depth`

For example:

Run 3
`max_depth = 3`

Run 4
`max_depth = 5`

Run 5
`max_depth = 8`

Now you're learning:

Same model
+
different hyperparameters

Step 4 — Experiment with class weights

Suppose the target is imbalanced.

Compare:

Run 6
`class_weight = None`

with:

Run 7
`class_weight = balanced`

Pay attention particularly to:

precision
recall
F1

rather than looking only at accuracy.

Step 5 — Change preprocessing

Try:

```
mean imputation
```

versus:

```
median imputation
```

or:

```
scaling
```

versus:

```
no scaling
```

Each meaningful change becomes another run.

Step 6 — Save artifacts

For useful runs, log:

```
confusion_matrix.png
```

```
predictions.csv
```

Now the run contains both:

```
numbers  
+  
visual/output evidence
```

Step 7 — Save the trained model

Record the fitted model with the run.

Conceptually:

```
Run 8  
  
├ Parameters  
├ Metrics  
└ Tags
```

└─ Confusion matrix
└─ Trained model

Step 8 — Add meaningful names/tags

Instead of:

```
Run 8
```

use something like:

```
tree_depth5_balanced
```

and perhaps:

```
purpose = class_weight_test
```

Step 9 — Compare everything in the UI

Your experiment could eventually look like:

Run	Model	Depth	Weight	Scaling	F1
logistic_base	Logistic	—	None	Yes	0.77
tree_base	Tree	None	None	No	0.79
tree_d3	Tree	3	None	No	0.81
tree_d5	Tree	5	None	No	0.83
tree_d5_bal	Tree	5	Balanced	No	0.87
rf_base	Random Forest	—	None	No	0.86

Do **not** focus only on finding the biggest number.

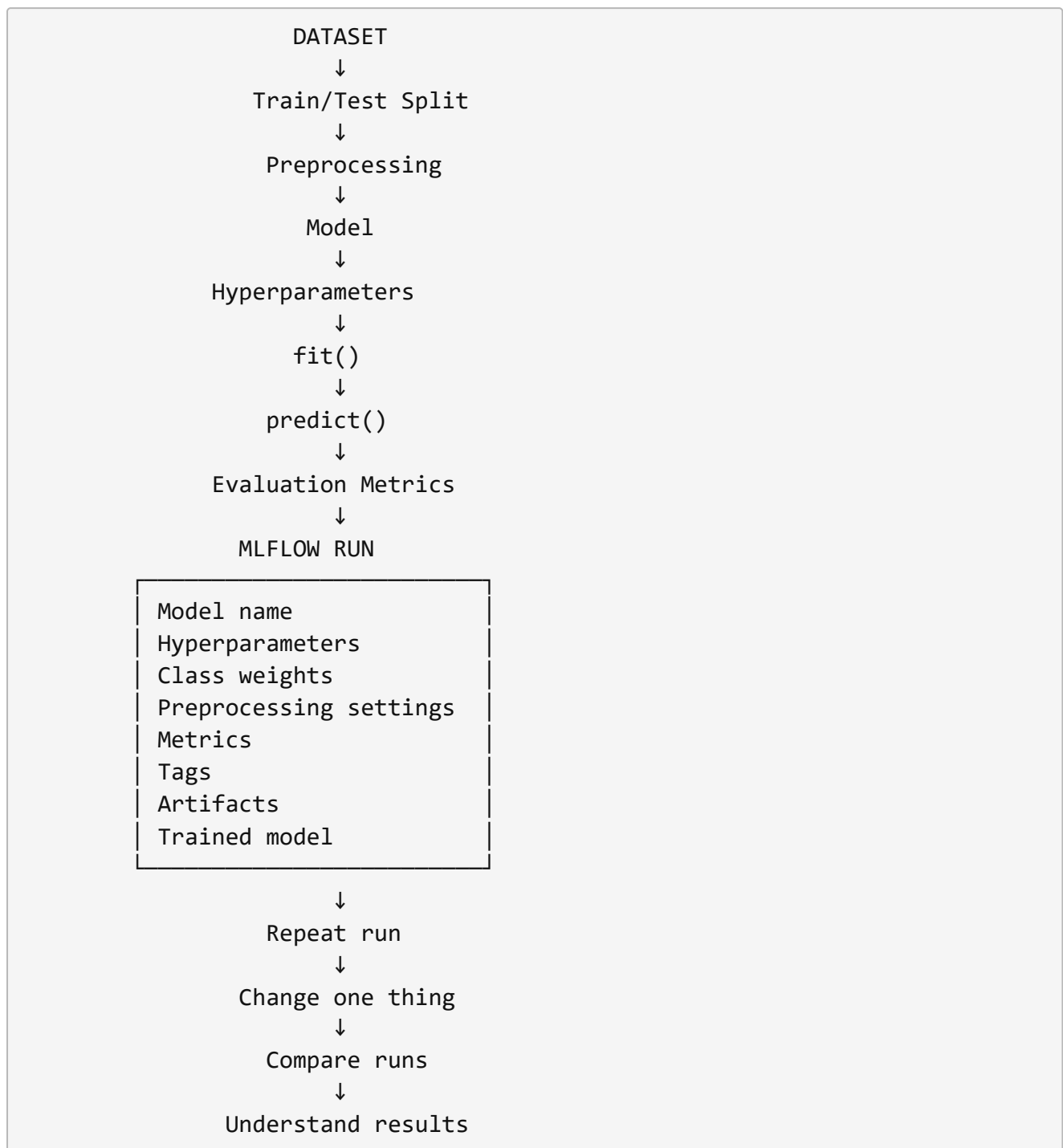
Ask:

```
What did I change?  
  
Why did I change it?  
  
Which metrics changed?  
  
Did one metric improve while another dropped?  
  
Can I reproduce this run?
```

That's the useful MLflow mindset.

Putting All 12 Topics Together

Your workflow now becomes:



What Counts as What?

This is worth memorizing:

Information	MLflow concept
<code>model = RandomForest</code>	Parameter/tag
<code>n_estimators = 100</code>	Hyperparameter / parameter

Information	MLflow concept
max_depth = 5	Hyperparameter / parameter
class_weight = balanced	Hyperparameter / parameter
scaler = StandardScaler	Preprocessing parameter/context
accuracy = 0.88	Metric
recall = 0.81	Metric
F1 = 0.84	Metric
purpose = baseline	Tag
confusion matrix image	Artifact
predictions CSV	Artifact
fitted Random Forest	Logged model

Beginner Thought Process

Whenever you're about to start another MLflow run, ask:

1. What am I changing?
2. Why am I changing it?
3. What should I log as a parameter?
4. Which metrics should I compare?
5. Should I save any useful artifacts?
6. Should I save the trained model?
7. Can I identify this run later?
8. Can I reproduce it?

If you can answer these questions, you're already using MLflow in a meaningful way.

Common Beginner Mistakes

Avoid treating MLflow as simply:

accuracy storage

The real value comes from recording the **context around an experiment**.

Also avoid changing five things at once:

```
model changed
+ preprocessing changed
+ class weights changed
+ dataset changed
+ hyperparameters changed
```

If the metric improves, you won't know which change caused it.

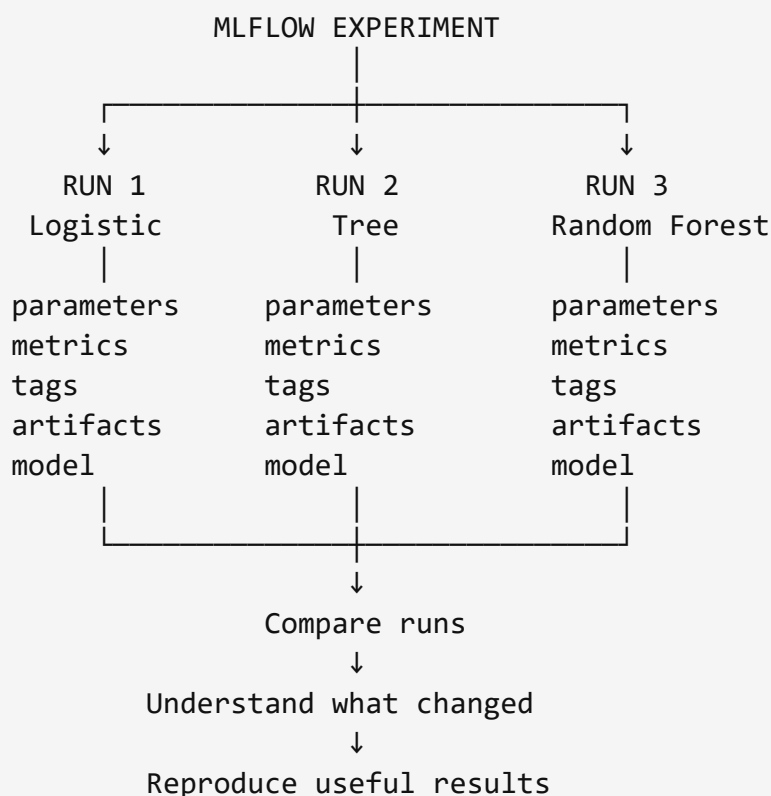
For learning, prefer:

```
Baseline
  ↓
change one important thing
  ↓
new run
  ↓
compare
```

Finally, remember that MLflow records experiments—it does not decide whether your ML methodology is correct. Data leakage, an inappropriate metric, or a bad train/test split can still produce misleading results even when perfectly logged.

Day 117 Part-3 Final Mental Model

By the end of Day 117 Part-3, think of MLflow like this:



The most important progression is:

Beginner ML

→ "I trained a model."

Better workflow

→ "I trained several models."

MLflow mindset

→ "I know exactly what I tried,
what settings I used,
what results I got,
and which files/models belong
to each experiment."

That covers the **12 practical MLflow topics** you should know at this stage. The next natural hands-on exercise is one tiny dataset with roughly **6–8 deliberately different MLflow runs**, rather than a large project.